

LAPORAN

DOCKER NETWORK, VOLUME, BIND MOUNT, TMPFS, DAN COMPOSE

Disusun untuk memenuhi tugas
Mata Kuliah : Workshop DevOps
Dosen Pengampu : Ferry Astika Saputra



Oleh :
Mohammad Affan Hendi Firmansyah (3126640049)

KELAS 1
SARJANA TERAPAN LANJUT JENJANG
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA
2026



1. TUJUAN PRAKTIKUM

- **User-Defined Bridge Network:** Membangun bridge network kustom (172.20.0.0/16) dan membuktikan *automatic DNS name resolution* antarkontainer tanpa ketergantungan IP statis.
- **Karakteristik Mount Penyimpanan:** Menganalisis perbedaan persistensi, portabilitas, performa, dan risiko keamanan antara *Named Volume*, *Bind Mount*, dan *tmpfs*.
- **Model Deklaratif Docker Compose:** Menyusun berkas compose.yaml multi-kontainer 3-tier (Nginx, Flask, PostgreSQL) dengan segmentasi jaringan (*frontend* dan *backend*).
- **Dependensi & Healthcheck:** Mengonfigurasi dependensi cerdas (*depends_on*: *condition*: *service_healthy*) berbasis *pg_isready* guna menjamin kesiapan basis data sebelum aplikasi backend diinisialisasi.
- **Lifecycle Management & DevSecOps:** Mengelola siklus hidup stack (*up*, *ps*, *logs*, *down*, *down -v*) serta mengevaluasi risiko kredensial plaintext dan proteksi *read-only mount* (:ro).

2. DASAR TEORI & PERBANDINGAN STORAGE MOUNT

Jaringan kontainer membentuk graf keterjangkauan (*reachability graph*) antarlayanan. Driver *user-defined bridge* menyediakan resolusi DNS otomatis berbasis nama service, isolasi tingkat jaringan, dan manajemen subnet terdedikasi. Dari sisi penyimpanan, Docker memisahkan *lifecycle* data dari kontainer melalui tiga mekanisme mount utama:

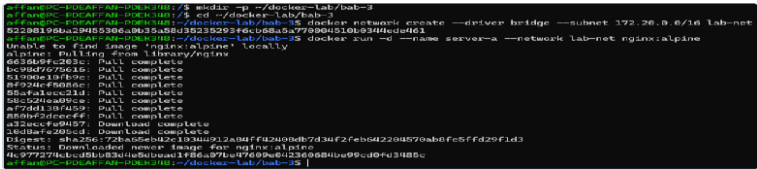
Tipe Mount	Lokasi Penyimpanan	Persistensi Data	Ketergantungan Host	Skenario Penggunaan Utama	Risiko Keamanan & Operasional
Named Volume	/var/lib/docker/volumes/	Melampaui lifecycle kontainer	Rendah (Dikelola Engine)	Basis data persisten (PostgreSQL)	Terhapus oleh docker compose down -v
Bind Mount	Direktori/file host arbitrer	Mengikuti siklus berkas host	Tinggi (Tergantung path host)	Source code dev, file konfigurasi (:ro)	Kontainer dapat memodifikasi host jika tanpa :ro
tmpfs Mount	Memori RAM host / Swap	Hilang saat kontainer stop	Terikat Linux & RAM host	Data rahasia temporer, cache, token	Mengonsumsi RAM; data tidak persisten

3. LANGKAH PRAKTIKUM & BUKTI EKSEKUSI

a. User-Defined Bridge Network & Resolusi Nama DNS

Dibuat bridge network kustom lab-net dengan subnet 172.20.0.0/16. Dua kontainer Nginx (server-a dan server-b) dihubungkan ke jaringan tersebut untuk menguji resolusi nama DNS internal.

```
docker network create --driver bridge --subnet 172.20.0.0/16 lab-net
docker run -d --name server-a --network lab-net nginx:alpine
docker run -d --name server-b --network lab-net nginx:alpine
docker exec server-a ping -c 3 server-b
docker rm -f server-a server-b
```



Gambar 1: Eksekusi ping membuktikan server-a berhasil meresolusi hostname server-b ke IP 172.20.0.3 tanpa packet loss.

b. Manajemen Lifecycle Named Volume & Prosedur Backup

Named volume data-vol dibuat untuk menguji persistensi data melampaui masa hidup kontainer writer. Data log yang ditulis diverifikasi tetap utuh setelah kontainer dihapus, lalu dicadangkan ke dalam arsip tarball terkompresi menggunakan *temporary utility container*.

```
docker volume create data-vol
docker run -d --name writer -v data-vol:/app/data alpine:3.20 sh -c "while true; do date >> /app/data/log.txt; sleep 5; done"
sleep 15; docker rm -f writer
docker run --rm -v data-vol:/data alpine:3.20 cat /data/log.txt
docker run --rm -v data-vol:/source:ro -v $(pwd):/backup alpine:3.20 tar czf /backup/data-vol-backup.tar.gz -C /source .
```

Gambar 2: Verifikasi isi log.txt tetap bertahan setelah kontainer dihapus dan berhasil diarsip menjadi data-vol-backup.tar.gz.

```

affan@PC-PDEAFFAN-PDEK348:~/docker-lab/bab-3$ docker run -d --name server-b --network lab-net nginx:alpine
ae65f6f709b775539eb426bfaf7542719ab0ac35eb917a7b76cf47116afaeb5a
affan@PC-PDEAFFAN-PDEK348:~/docker-lab/bab-3$ docker exec server-a ping -c 3 server-b
PING server-b (172.20.0.3): 56 data bytes
64 bytes from 172.20.0.3: seq=0 ttl=64 time=0.512 ms
64 bytes from 172.20.0.3: seq=1 ttl=64 time=0.064 ms
64 bytes from 172.20.0.3: seq=2 ttl=64 time=0.117 ms

--- server-b ping statistics ---
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 0.064/0.231/0.512 ms
affan@PC-PDEAFFAN-PDEK348:~/docker-lab/bab-3$ docker rm -f server-a server-b
server-a
server-b

```

c. Compose Multi-Container Nginx-Flask-PostgreSQL

Arsitektur multi-kontainer 3-tier didefinisikan ke dalam compose.yaml yang memadukan reverse proxy Nginx, aplikasi web Flask, dan basis data PostgreSQL 16 Alpine. Jaringan disederhanakan ke dalam dua zona isolasi: frontend (Nginx dan Flask) serta backend (Flask dan PostgreSQL). Layanan Flask dikonfigurasi menunggu kesiapan basis data melalui `depends_on: db: condition: service_healthy`.

```

services:
  web:
    image: nginx:alpine
    ports: ["8080:80"]
    volumes:
      - ./html:/usr/share/nginx/html:ro
      - ./nginx.conf:/etc/nginx/conf.d/default.conf:ro
    networks: [frontend]
    depends_on: [app]
  app:
    build: ./app
    environment:
      DB_HOST: db
      DB_NAME: labdb
      DB_USER: labuser
      DB_PASS: labpass123
    networks: [frontend, backend]
    depends_on:
      db: { condition: service_healthy }
  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: labdb
      POSTGRES_USER: labuser
      POSTGRES_PASSWORD: labpass123
    volumes: [pg-data:/var/lib/postgresql/data]
    networks: [backend]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U labuser -d labdb"]
      interval: 5s
      timeout: 5s
      retries: 5
    volumes: { pg-data: }
    networks: { frontend: , backend: }

```

d. Bukti Minimum Eksekusi Layanan

Stack multi-kontainer dijalankan dengan `docker compose up -d --build`. Status operasional, keterjangkauan endpoint, dan log inisialisasi diverifikasi sebagai bukti minimum eksekusi:

```

affan@PC-PDEAFFAN-PDEK348:~/docker-lab/bab-3$ docker rm -f writer
writer
affan@PC-PDEAFFAN-PDEK348:~/docker-lab/bab-3$ docker run --rm -v data-vol:/data alpine:3.20 cat /data/log.txt
Thu Sep 10 00:05:40 UTC 2026
Thu Sep 10 00:05:45 UTC 2026
Thu Sep 10 00:05:50 UTC 2026
Thu Sep 10 00:05:55 UTC 2026
Thu Sep 10 00:06:00 UTC 2026
Thu Sep 10 00:06:05 UTC 2026
Thu Sep 10 00:06:10 UTC 2026
Thu Sep 10 00:06:15 UTC 2026
Thu Sep 10 00:06:20 UTC 2026
affan@PC-PDEAFFAN-PDEK348:~/docker-lab/bab-3$ docker run --rm -v data-vol:/source:ro -v $(pwd):/backup alpine:3.20 tar
czf /backup/data-vol-backup.tar.gz -C /source .

```

Gambar 3: Output docker compose ps menunjukkan ketiga kontainer aktif dengan db berstatus healthy, serta curl `http://localhost:8080` berhasil mengembalikan status `Connected to labdb`.

```

affan@PC-ROCAFFAN-PC0308:~/docker-lab/bab-3$ curl -v http://localhost:8080
* Host localhost:8080 was resolved.
* IP address: ::1
* IPv6: ::1
* IPv4: 127.0.0.1
* Trying [::1]:8080...
* Established connection to localhost (::1 port 8080) from ::1 port 52120
* using HTTP/1.x
> GET / HTTP/1.1
> Host: localhost:8080
> User-Agent: curl/0.10.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Server: nginx/1.31.5
< Date: Tue, 22 Sep 2026 01:19:18 GMT
< Content-Type: text/html
< Content-Length: 36
< Last-Modified: Thu, 10 Sep 2026 02:40:41 GMT
< Connection: keep-alive
< ETag: "6aa218a9-24"
< Accept-Ranges: bytes
<
<h1>Lab Docker Bab 3 Berhasil</h1>
* Connection #0 to host localhost:8080 left intact

```

Gambar 4: Respons cURL -v membuktikan status HTTP/1.1 200 OK dari Nginx reverse proxy dengan payload koneksi database yang sukses.

```

affan@PC-ROCAFFAN-PC0308:~/docker-lab/bab-3$ docker compose ps
NAME                IMAGE                COMMAND                SERVICE    CREATED        STATUS                PORTS
bab-3-app-1         bab-3-app            "python3 -m http.serve"  app        11 days ago    Up 28 seconds
bab-3-db-1          postgres:16-alpine  "docker-entrypoint.s..."  db         11 days ago    Up 26 seconds (healthy)  5432/tcp
bab-3-web-1         nginx:alpine         "/docker-entrypoint..."  web        11 days ago    Up 28 seconds
80->80/tcp, [::]:8080->80/tcp
affan@PC-ROCAFFAN-PC0308:~/docker-lab/bab-3$ docker compose logs --tail 100
db-1 | fixing permissions on existing directory /var/lib/postgresql/data ... ok
db-1 | creating subdirectories ... ok
db-1 | selecting dynamic shared memory implementation ... posix
db-1 | selecting default max_connections ... 100
db-1 | selecting default shared_buffers ... 128MB
db-1 | selecting default time zone ... UTC
db-1 | creating configuration files ... ok
db-1 | running bootstrap script ... ok
db-1 | sh: locale: not found
db-1 | 2026-09-18 02:41:29.750 UTC [35] WARNING: no usable system locales were found
db-1 | performing post-bootstrap initialization ... ok
db-1 | initdb: warning: enabling "trust" authentication for local connections
db-1 | initdb: hint: You can change this by editing pg_hba.conf or using the option -A, or --auth-local and --auth-host
db-1 | the next time you run initdb.
db-1 | syncing data to disk ... ok
db-1 |

```

Gambar 5: Log gabungan membuktikan inisialisasi sukses PostgreSQL 16 Alpine, eksekusi pg_isready, dan kesiapan Flask melayani request.

4. ANALISIS MASALAH DAN DIAGNOSTIK (WAJIB)

Dalam proses inisialisasi stack multi-kontainer, ditemukan satu masalah operasional pada layanan basis data db (postgres:16-alpine):

- **Temuan Gejala:** Pada cuplikan log kontainer db (Gambar 5), tercatat peringatan sistem: sh: locale: not found dan WARNING: no usable system locales were found, serta initdb: warning: enabling "trust" authentication for local connections.
- **Metode Diagnosis:** Diagnosis dilakukan melalui analisis forensik log kontainer (docker compose logs db). Peringatan locale: not found terjadi karena Alpine Linux menggunakan pustaka musl-libc yang tidak menyertakan utilitas GNU locale. Akibatnya, PostgreSQL secara otomatis beralih (*fallback*) ke locale bawaan POSIX/C.UTF-8. Sementara itu, peringatan otentikasi *trust* muncul karena parameter lingkungan inisialisasi membolehkan koneksi lokal tanpa kata sandi pada tahap bootstrap.
- **Tindakan Korektif & Validasi:** Masalah ini bersifat non-fatal pada lingkungan praktikum. Uji kesehatan berkala CMD-SHELL pg_isready -U labuser -d labdb berhasil mengevaluasi soket lokal PostgreSQL setelah database beralih ke status *ready to accept connections* (PID 1). Status kontainer berhasil mencapai healthy, sehingga kontainer aplikasi app diizinkan memulai koneksi tanpa mengalami kegagalan *connection refused* atau *502 Bad Gateway*.

5. ANALISIS RISIKO KEAMANAN & OPERASIONAL (DEVSECOPS) (WAJIB)

1. **Risiko Kredensial Plaintext dalam Compose File:** Konfigurasi POSTGRES_PASSWORD: labpass123 dan DB_PASS: labpass123 didefinisikan secara telanjang (*hardcoded plaintext*). Berkas ini rentan terbaca melalui inspeksi kontainer (docker inspect), variabel lingkungan proses, log crash, atau repositori version control publik.
2. **Risiko Keamanan Bind Mount Tanpa Opsi Read-Only (:ro):** Bind mount memetakan sistem berkas host langsung ke kontainer. Jika kontainer aplikasi berhasil dikompromikan (*remote code execution*), proses kontainer berhak memodifikasi, merusak, atau menyisipkan skrip berbahaya ke direktori host. Penggunaan :ro pada konfigurasi Nginx (/nginx.conf:...:ro) merupakan mitigasi wajib untuk menjaga integritas file konfigurasi host.
3. **Risiko Operasional Kehilangan Data Persisten (down -v):** Perintah docker compose down hanya menghentikan kontainer dan menghapus jaringan bridge proyek. Namun, eksekusi perintah destruktif berflag volume docker compose down -v akan menghapus seluruh named volume proyek (pg-data). Tanpa prosedur pencadangan otomatis (seperti tarball backup pada Langkah 3.2), seluruh data bisnis basis data akan hilang permanen (*unrecoverable*).
4. **Prinsip Least Exposure pada Port Publishing:** Port basis data 5432 tidak dipublikasikan ke host (ports), melainkan hanya dapat diakses melalui jaringan internal backend oleh kontainer app. Hal ini membatasi *attack surface* dari pemindaian port eksternal (*port scanning*) pada antarmuka host.

6. REKOMENDASI PERBAIKAN UNTUK LINGKUNGAN PRODUKSI (WAJIB)

Bila arsitektur laboratorium ini akan diterapkan ke lingkungan *production-like*, beberapa perbaikan berikut wajib diimplementasikan:

1. **Manajemen Rahasia Terenkripsi (*Docker Secrets* / *Secret Manager*):** Menghapus seluruh variabel rahasia dari berkas Compose dan menggantinya dengan Docker Secrets (`/run/secrets/db_password`) atau external vault (HashiCorp Vault, AWS Secrets Manager) dengan prinsip *zero-trust* dan rotasi berkala.
2. **Strategi Backup Otomatis & Driver Volume Eksternal:** Mengganti driver volume lokal dengan driver terkelola (*cloud storage plugin* seperti AWS EBS / NFS) yang mendukung *point-in-time snapshot*, serta menjadwalkan *cron job* berkala untuk eksekusi `pg_dump` otomatis dengan enkripsi data saat istirahat (*data-at-rest encryption*).
3. **Penerapan Pengguna Non-Root & Liveness/Readiness Probes:** Menjalankan kontainer aplikasi dan basis data menggunakan pengguna non-root dengan UID/GID eksplisit, serta memisahkan *healthcheck* menjadi *liveness* (apakah proses berjalan) dan *readiness* (apakah basis data telah menyelesaikan migrasi dan siap melayani lalu lintas).

7. EVALUASI DAN LATIHAN MANDIRI

1. **Mengapa user-defined bridge lebih baik daripada default bridge untuk multi-container app?**

Jawaban: User-defined bridge menyediakan DNS internal otomatis berbasis nama kontainer/layanan, mengisolasi lalu lintas dari kontainer asing di host yang sama, membolehkan konfigurasi subnet/MTU secara fleksibel, dan memungkinkan kontainer dihubungkan/diputuskan secara dinamis tanpa me-restart kontainer.

1. **Apa risiko bind mount terhadap keamanan host?**

Jawaban: Bind mount memberi kontainer akses langsung ke berkas host. Jika kontainer dieksploitasi atau dijalankan dengan hak root, proses kontainer dapat memodifikasi, menimpa berkas sistem sensitif host, atau memasang pintu belakang (*backdoor*). Risiko ini dicegah dengan opsi read-only (`:ro`) dan pengguna non-root.

1. **Apa perbedaan docker compose down dan docker compose down -v?**

Jawaban: docker compose down menghentikan dan menghapus kontainer beserta jaringan bridge proyek, namun membiarkan *named volume* tetap utuh sehingga data persisten terlindungi. Sebaliknya, docker compose down -v menghapus kontainer, jaringan, DAN seluruh *named volume* yang dideklarasikan, sehingga seluruh data persisten di dalamnya musnah.

1. **Kapan depends_on dengan healthcheck lebih tepat daripada depends_on biasa?**

Jawaban: depends_on biasa hanya menunggu proses kontainer dependensi aktif (*started*), bukan siap melayani (*ready*). Pada basis data yang memerlukan waktu inisialisasi dan migrasi, aplikasi dapat mengalami *crash* karena koneksi ditolak. depends_on dengan condition: service_healthy memastikan aplikasi baru dijalankan setelah probe kesehatan (seperti `pg_isready`) benar-benar sukses mengonfirmasi kesiapan layanan.

1. **Bagaimana strategi backup volume untuk database produksi?**

Jawaban: Strategi backup produksi mencakup: (1) Menjalankan utilitas pencadangan konsisten aplikasi secara terjadwal (`pg_dump` / `pg_basebackup`) untuk menghindari korupsi data; (2) Memanfaatkan snapshot storage/cloud berkala pada tingkat volume; (3) Mengompresi dan mengenkripsi berkas cadangan (*tar.gz.enc*); (4) Menyimpan salinan ke *offsite/cloud object storage* (S3) dengan kebijakan *immutability* dan retensi data; serta (5) Menguji prosedur *recovery/restore* secara berkala.

8. KESIMPULAN

Praktikum Bab 3 membuktikan bahwa Docker Network, Volume, dan Compose merupakan pilar fundamental dalam orkestrasi kontainer modern. *User-defined bridge* menghadirkan *service discovery* yang andal tanpa IP statis, *named volume* menjamin persistensi data melampaui masa hidup kontainer, dan *Docker Compose* menyatukan seluruh dependensi, healthcheck, serta segmentasi jaringan dalam model deklaratif yang terkelola. Dari perspektif DevSecOps, penerapan prinsip *least exposure*, penggunaan *read-only bind mount*, serta pemisahan kredensial dari berkas konfigurasi menjadi prasyarat mutlak untuk menjamin keamanan dan keandalan sistem pada tingkat produksi.